

Elfin Twins GM-021

Technical Reference

Hardware, firmware and game logic of a 1990s virtual pet,
and the architecture of its Game Boy Advance port



Reconstructed from the device ROM (`assets/ElfinTwins.bin`, 32 KB) with BrickEmuPy, a C port of its SPLB20 core, an analysis disassembler and instrumented emulation.

Companion files: `gba/docs/analysis/elfin_twins.asm` (annotated disassembly of the whole ROM), `gba/tools/elfin_symbols.py` (symbol table), `gba/host/*` (verification and analysis tools).

September 2026.

Contents

1 Scope, sources and method

2 Hardware

2.1 Microcontroller and memory map

2.2 Special function registers

2.3 Interrupts, sleep and wake-up

2.4 Timers and buzzer

2.5 Port A: buttons, link line, buzzer

2.6 LCD

2.7 Instruction set

3 Firmware architecture

3.1 Control flow

3.2 Timekeeping

3.3 Key decoder

3.4 Game modes and dispatch tables

3.5 Menu and icons

3.6 Graphics, animation and sound engines

4 Game state (RAM map)

5 Game rules

5.1 Statistics

5.2 Periodic rules

5.3 Care actions

5.4 Day, night and locations

5.5 Life cycle

5.6 Password, clock stop, test mode

6 Link protocol

7 Game Boy Advance port

7.1 Overview

7.2 Core port and verification

7.3 Performance

7.4 Display, input, sound

7.5 Link bridge

7.6 RTC catch-up

7.7 Saving

7.8 Build and test tooling

8 Provenance: who made it

8.1 The hidden credit

8.2 Sunplus factory test program

8.3 Other toys, and what is not known

9 Open questions

A Appendix: routine index

B Appendix: LCD mapping

C Appendix: instruction set

1 Scope, sources and method

This document describes the Elfin Twins GM-021 handheld virtual pet at three levels: the hardware as seen by its program, the firmware's structure, and the game rules it implements. It also documents the Game Boy Advance port built on top of that knowledge. No original documentation was available. Everything was derived from the 32 KB ROM dump distributed with BrickEmuPy, whose Python emulator core (`cores/SPLB20.py`) and device description (`assets/ElfinTwins.brick`, `.svg`) were the starting point.

The method combined four tools, all in the repository:

- **A C port of the core**, checked instruction by instruction against the Python core, used for fast experiments (hours of pet time per second).
- **An analysis disassembler** (`gba/tools/elfin_disasm.py`) that follows the ROM's `pha/pha/rt`s jump tables and annotates the code with the symbols of `elfin_symbols.py`. 15430 bytes of code reachable from the vectors and jump tables (47.1% of the ROM), 172 subroutines, 8 jump tables; the rest is graphics, animation scripts, melodies and lookup tables.
- **A scripted workbench** (`gba/host/elfin_sim.py`, `explore.py`) to press buttons, poke RAM, record LCD frames and trace calls of chosen routines (for example the stat routines) with the game mode in effect.
- **A two-unit link simulator** (`gba/host/link_sim.py`) that wires two emulated toys together.

Each statement about the game is tagged with how it is known: `code` read in the disassembly, `observed` in instrumented emulation, `inferred` from indirect evidence (for example a name guessed from an effect or a picture).

2 Hardware

2.1 Microcontroller and memory map

The toy is built around a Sunplus **SPLB20**, a 4-bit-era LCD controller with a reduced 6502 CPU (70 opcodes, no Y-indexed modes, no stack page: the stack lives in zero page). BrickEmuPy runs it at **560 kHz** from its RC oscillator with a 32 768 Hz sub-clock for timekeeping. The 64 KB address space is laid out as follows

code :

Range	Contents
<code>\$0000-\$003F</code>	LCD RAM, 64 bytes (512 segment bits, 480 used)
<code>\$0040-\$007F</code>	special function registers (only <code>\$70-\$7F</code> implemented)
<code>\$0080-\$00FF</code>	CPU RAM, 128 bytes: game state and the stack (SP starts at <code>\$FF</code>)
<code>\$0200-\$09FF</code>	data RAM, 2 KB, mirrored (e.g. at <code>\$1000</code>). The game uses <code>\$1080-\$10AF</code> for the name, password and quiz data.
<code>\$8000-\$FFFF</code>	ROM, 32 KB. Vectors: NMI <code>\$8C01</code> , RESET <code>\$970C</code> , IRQ <code>\$8C00</code> (a bare <code>rti</code>). <code>\$8000-\$83D4</code> is Sunplus's factory test program, entered through test vectors at <code>\$FFF2-\$FFF7</code> (§8.2). <code>\$FF27</code> holds the developer credit (§8.1).

2.2 Special function registers

Addr	Name	Function
<code>\$70</code>	<code>LCD_CFG</code>	LCD configuration
<code>\$71</code>	<code>PA_DIR</code>	port A direction: 1 = input, 0 = output
<code>\$72</code>	<code>PA_CFG</code>	port A output config; bits 6-7 = buzzer
<code>\$73</code>	<code>PA_DATA</code>	port A: b0 ESC b1 LEFT b2 RIGHT b3 CLOCK b4 ENTER (0 = pressed), b5 LINK
<code>\$74</code>	<code>PXY_DATA</code>	port X/Y (unused)
<code>\$75</code>	<code>PZ_DATA</code>	port Z (unused)
<code>\$76</code>	<code>LCD_BIAS</code>	LCD bias / contrast
<code>\$77</code>	<code>KEYSCAN_L</code>	key scan output (unused)
<code>\$78</code>	<code>KEYSCAN_H</code>	key scan output (unused)
<code>\$79</code>	<code>INT_CFG</code>	interrupt enable (write) / pending flags (read, clears)
<code>\$7A</code>	<code>SYS_CTRL</code>	system control: b7 stop osc, b5 32k, b4 timer, b3 LCD on, b2 LCD en
<code>\$7B</code>	<code>TC_PRESET</code>	down-counter preset = buzzer pitch
<code>\$7C</code>	<code>PRESCALER</code>	timer prescaler (1 << n)
<code>\$7E</code>	<code>KEYSCAN_CTRL</code>	key scan control
<code>\$7F</code>	<code>WDT_CLR</code>	watchdog clear

Two register behaviours matter for emulation. Reading `INT_CFG` returns the pending flags and clears them. The stop bits of `SYS_CTRL` act on a 0→1 transition: writing `$AD` while bit 7 is already set does not stop the oscillator. That is why the wake-up path always writes `$2C` first.

2.3 Interrupts, sleep and wake-up

All interrupts are delivered as **NMI**, gated by bit 7 of `INT_CFG`. Sources: 2 Hz tick (b0), 128 Hz tick (b1), power key PA0 (b2), any other port-A key (b3), down-counter underflow (b4). The IRQ vector is unused. The firmware spends about 90% of the time **asleep**. At `$8E7A` it loads `INT_CFG` from `int_sleep` (normally `$8D`: NMI, both keys, 2 Hz) and writes `SYS_CTRL=$AD`, stopping the main oscillator while the 32 kHz clock and the LCD keep running. An interrupt during STOP does not enter the NMI handler: the chip restarts at the **RESET** vector, and `$970C` uses the pending flags (`$79` & `$8F`) to tell a wake-up from a cold start [code](#).

RESET \$970C: `SYS_CTRL=$2C`, `SP=$FF`, `irq_flags = INT_CFG`. Any flag set → **wake \$8EB2**. Otherwise cold start: clear RAM, hardware init, LCD test if a key is held, else **new_pair \$9827**.



wake \$8EB2: key flag and PA5 low → link handling (`$8EC2`). 2 Hz flag → **tick_2hz**. Then `rnd_next`, key path (debounce → **key_decode \$8FB8** → **key_dispatch** `T_959D[mode]`) or tick path (**tick_dispatch** `T_957F[mode]`).



sleep_main \$8E74: update ?! (**attention \$8E26**), wait for the melody (**wait_melody \$C626**), `INT_CFG=int_sleep`, `SYS_CTRL=$AD` → STOP until the next interrupt.

2.4 Timers and buzzer

The 2 Hz and 128 Hz ticks are derived from the 32 kHz clock (280 000 and 4 375 CPU cycles at 560 kHz). The 8-bit down counter is clocked at $\text{clk} / 2^{\text{PRESCALER}}$ and reloads from `TC_PRESET`. It also drives the buzzer, which sounds when `PA_CFG` bits 6–7 are set and the timer is enabled:

```
f_buzzer = 560000 / 2^PRESCALER / (TC_PRESET + 1)
```

Melodies are note lists (duration, preset) walked by the NMI handler on the 128 Hz tick through `snd_len/snd_ptr`. `snd_play` starts melody X from a pointer table at `$F7D5/$F7EB` [code](#).

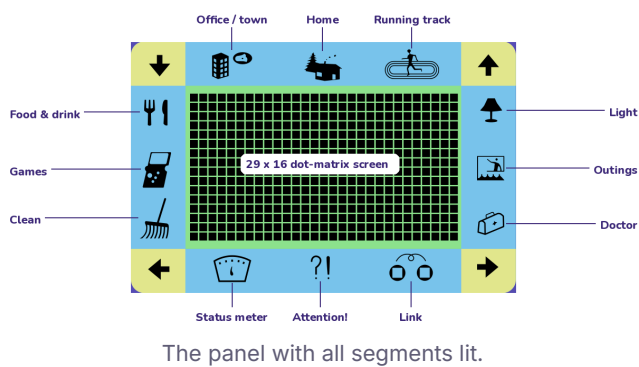
2.5 Port A: buttons, link line, buzzer

Bit	Use	Notes
0	ESC	the "power key" interrupt source
1	LEFT	active low, internal/external pull-ups (mask option 255)
2	RIGHT	
3	CLOCK	
4	ENTER	
5	LINK	open-drain link wire; input normally, output-low to transmit
6–7	buzzer	via <code>PA_CFG</code>

The toy has seven buttons but only five contacts. **UP** closes **LEFT+RIGHT** and **DOWN** closes **CLOCK+ENTER** (`PA&$1F` = `$19` and `$07`) [code](#). BrickEmuPy's `ElfinTwins.brick` originally wired DOWN to RIGHT+CLOCK (`$13`), which the decoder ignores. The copy of `ElfinTwins.brick` in this repository uses the corrected mask 24.

2.6 LCD

480 of the 512 LCD RAM bits are used: a **29 × 16 dot matrix** (464 bits) and **16 icons** printed on the panel. The bit order is irregular (Appendix B). The firmware draws with a bitmap blitter (`gfx_draw $9061`) and addresses icons through a table of (byte, mask) pairs at `$9E26` [code](#).



Icon	LCD byte	Bit	Mask
arrow_down	\$2F	0	\$01
arrow_left	\$17	0	\$01
arrow_right	\$0C	6	\$40
arrow_up	\$04	6	\$40
bag	\$34	6	\$40
building	\$0F	0	\$01
clean	\$27	0	\$01
exercise	\$24	6	\$40
food	\$37	0	\$01
game	\$3F	0	\$01
house	\$07	0	\$01
lamp	\$2C	6	\$40
link	\$14	6	\$40
meter	\$1F	0	\$01
picture	\$3C	6	\$40
question	\$1C	6	\$40

2.7 Instruction set

The core implements 70 opcodes of the 6502 with standard encodings and 6502-like cycle counts; the other 186 opcodes are illegal. Decimal mode exists (`sed`) but the game does not rely on it. See Appendix C.

3 Firmware architecture

3.1 Control flow

The program is event-driven around the sleep/wake cycle of §2.3. Nothing runs in a main loop. Every piece of work starts from a wake-up: the 2 Hz tick, a key, or an edge on the link line. The NMI handler itself (`$8C01`) only runs while the CPU is awake, for example during animations and delays. It performs the 2 Hz tick and steps the melody engine on the 128 Hz tick. Long animations are therefore written as sequences of short steps, one per wake-up, with their progress kept in RAM (`sub_sel`, `$C0`, `$C1` ...) `code` .

3.2 Timekeeping

`tick_2hz` `$8C6D` keeps two independent sets of counters `code` :

Counter	Range	On overflow
clock: <code>clk_halfsec</code> → <code>clk_min</code> → <code>clk_hour</code>	120 / 60 / 24	—
life: <code>life_halfsec</code> → <code>life_min</code>	120 / 10	every 10 minutes: <code>life_10min++</code> , <code>night_flag</code> , the 10-minute rules (§5.2)
<code>life_10min</code>	144	every 24 h: <code>age_days++</code> , <code>growth_stage</code> , stats -1, lifespan check, <code>birthday</code>

The clock can be set and stopped by the player, while the life timer only counts. So a pair's birthday is 24 hours after it arrived, whatever the clock says. On 118 of 120 ticks the routine only increments `clk_halfsec` and `life_halfsec`, which is what makes fast catch-up possible (§7.6).

3.3 Key decoder

`key_decode` `$8FB8` compares `PA&$1F` with the value seen when the key went down (`key_raw`) on each 128 Hz wake-up and counts in `key_hold`. Most keys are accepted after 4 identical samples (31 ms). ENTER, ESC and CLOCK also have a long form after 255 samples (2 s) `code` :

PA&\$1F	Contacts	Code	Name	PA&\$1F	Contacts	Code	Name
\$19	LEFT+RIGHT	1	UP	\$0F	ENTER	5 / 7	ENTER / long
\$07	CLOCK+ENTER	2	DOWN	\$1E	ESC	6 / 9	ESC / long
\$1D	LEFT	3	LEFT	\$17	CLOCK	8 / B	CLOCK / long
\$1B	RIGHT	4	RIGHT	\$0E	ESC+ENTER	A	rescue / reset

3.4 Game modes and dispatch tables

`mode` (`$BD`) holds the current screen in its low nibble and a sub-state in its high nibble. Two parallel 15-entry jump tables are indexed by the low nibble `code` :

Mode	Meaning	Tick handler (T_957F)	Key handler (T_959D)
\$0	Home (normal life)	<code>\$98E8 home_tick</code>	<code>\$9AF6 home_key</code>
\$1	Food & drink	<code>\$A007 food_tick</code>	<code>\$A07C food_key</code>
\$2	Light	<code>\$A287 lamp_tick</code>	<code>\$A305</code>
\$3	Games	<code>\$A386 game_tick</code>	<code>\$A40A game_key</code>
\$4	Doctor / hospital	<code>\$AA8E doctor_tick</code>	<code>\$AAEA</code>
\$5	Clean	<code>\$AAED clean_tick</code>	<code>\$AB37</code>
\$6	Status meter	<code>\$AB3A meter_tick</code>	<code>\$AB84 meter_key</code>
\$7	Outings	<code>\$AFAD outing_tick</code>	<code>\$B0B9 outing_key</code>
\$8	Asleep (lights off)	<code>\$BF12 sleep_tick</code>	<code>\$BF7F</code>
\$9	Birthday	<code>\$BFED birthday_tick</code>	<code>\$C012</code>
\$A	Collapsed	<code>\$C015 collapse_tick</code>	<code>\$C09B collapse_key</code>
\$B	Farewell (end of life)	<code>\$C103 ending_tick</code>	<code>\$C140</code>
\$C	Opening story + SIGN NAME	<code>\$C2CB intro_tick</code>	<code>\$C1C4</code>
\$D	Password	<code>\$95BB password_tick</code>	<code>\$9631 password_key</code>
\$E	Link session	<code>\$B76C link_tick</code>	<code>\$B7ED link_key</code>

Modes 1–5, 7 and E return home after 240 ticks (2 minutes) without a key (`idle_ticks`). Any mode can be interrupted by the clock screen (`flags_b9` bit 7) and by the password prompt.

3.5 Menu and icons

The menu cursor `menu_sel` is a one-hot byte that LEFT/RIGHT rotate. ENTER converts it to a mode through the table at `$9CC2 = 02 07 04 0E 06 05 03 01` (indexed by the bit position, counted from the MSB): lamp→2, outing→7, doctor→4, link→E, meter→6, clean→5, game→3, food→1. ENTER only works while the cursor icon is shown (`flags_b7` bit 2) and not during a "busy" animation (`flags_b9` bit 2). The icon table at `$9E26` lists, in order: the 8 menu icons, ?!, the three locations and the four arrows `code` .

3.6 Graphics, animation and sound engines

`gfx_draw` copies a bitmap from ROM (`gfx_ptr`) into LCD RAM at (`gfx_x`, `gfx_y`). Scenes are started with `anim_start` `$C641` using an id in `anim_id` (for example `$4C` birthday, `$4E` sleepy, `$80` mess, `$82` ill, `$52`–

\$56 link games). Idle life at home is a random choice of short scenes from per-location, per-stage tables at \$9AD3/\$9AD9/\$9ADF. About 55% of the ROM is data: bitmaps, scene scripts, melodies and lookup tables `code`.

4 Game state (RAM map)

All game state lives in the 128 bytes of CPU RAM. The stack grows down from \$FF and uses about 16 bytes. Names are those of `elfin_symbols.py`. Flags are only listed where their meaning is established.

Addr	Name	Meaning
\$80	<code>lifespan</code>	days the pair will stay: 23-26, chosen at birth (rnd & 3) + 23
\$81	<code>rnd</code>	random number, advanced by <code>rnd_next</code>
\$82	<code>clk_halfsec</code>	clock: half-seconds 0-119
\$83	<code>clk_hour</code>	clock: hour 0-23
\$84	<code>clk_min</code>	clock: minute 0-59
\$85	<code>irq_flags</code>	INT_CFG pending flags latched at wake-up
\$86	<code>irq_nmi</code>	pending flags seen by the NMI handler
\$87	<code>int_sleep</code>	INT_CFG value used while asleep (\$8D normally, \$8C = CLOCK STOP)
\$88	<code>life_10min</code>	life timer: 10-minute blocks of the current day, 0-143
\$89	<code>life_min</code>	life timer: minutes 0-9
\$8A	<code>life_halfsec</code>	life timer: half-seconds 0-119
\$8B	<code>idle_ticks</code>	2 Hz ticks without input in a sub-screen (240 = back home)
\$8C	<code>weight</code>	weight in kg, 8-99
\$8D	<code>age_days</code>	age in days ('YR' on the meter), starts at 1
\$8E	<code>st_mood</code>	mood (flowers page) 0-33, shown x3
\$8F	<code>st_iq</code>	IQ 0-33, shown x3
\$90	<code>st_stamina</code>	stamina (sneaker page) 0-33, shown x3
\$91	<code>st_social</code>	friendship (people page) 0-33, shown x3
\$92	<code>money</code>	money in \$100 units, 0-99 (0 = collapse)
\$93	<code>tummy</code>	meals in the tummy 0-5 (0 = hungry)
\$94	<code>drinks</code>	drinks 0-5 (0 = thirsty)
\$95	<code>sickness</code>	sickness 0-10: >=5 ill, >=10 collapse
\$96	<code>dirt</code>	dirt from digested meals: >=2 needs a bath, >=3 costs money
\$97	<code>care_clock</code>	10-minute counter since wake-up driving hunger/thirst/digestion
\$98	<code>ill_clock</code>	10-minute counter while ill (6 = one hour)
\$9A	<code>snd_len</code>	melody: remaining duration of the current note
\$9B	<code>snd_ptr</code>	melody: pointer to the note list (lo)
\$9C	<code>snd_ptr_hi</code>	melody: pointer to the note list (hi)
\$9D	<code>key</code>	decoded key: 1 UP 2 DOWN 3 LEFT 4 RIGHT 5 ENTER 6 ESC 7 ENTER-long 8 CLOCK 9 ESC-long A ESC+ENTER B CLOCK-long
\$9E	<code>key_raw</code>	PA_DATA & \$1F when the key went down
\$9F	<code>key_hold</code>	key debounce / hold counter (4 = pressed, 255 = long)
\$A0	<code>gfx_x</code>	sprite draw: x position
\$A1	<code>gfx_y</code>	sprite draw: y position
\$A2	<code>gfx_ptr</code>	sprite draw: pointer to bitmap (lo)
\$A3	<code>gfx_ptr_hi</code>	sprite draw: pointer to bitmap (hi)

Addr	Name	Meaning
\$AE	tmp_count	temp; pulse count for link_send
\$AF	tmp	temp (also the level written by link_send!)
\$B4	link_edges	link: edges counted by link_rcv; also temp
\$B5	link_state	link session state (b7 = we are the caller)
\$B6	idle_secs	seconds since the last key press (stops at \$F0)
\$B7	flags_b7	b1 ?! icon on, b2 menu cursor shown, b3 melody playing, b6 sound off, b7 ...
\$B8	flags_b8	b0 locked (password), b1 TEST MODE, b2 night (22-07), b3 screen initialised, ...
\$B9	flags_b9	b0 CLOCK STOP, b2 busy animation, b3 password set, b6 clock setting, b7 clock screen
\$BA	sub_sel	sub-screen choice / animation step
\$BB	life_bits	b0-1 growth stage, b2 choosing destination, b3 room messy, b4-5 destination, b6-7 location
\$BC	menu_sel	menu cursor, one bit: 1 food 2 game 4 clean 8 meter 10 link 20 bag 40 outing 80 lamp
\$BD	mode	game mode (low nibble) + sub-state (high nibble)
\$C4	anim_id	animation / scene id for anim_start
\$CA	sel2	secondary selection (food, location)
\$CB	mode_saved	mode to return to after an interruption
\$CC	collapse_10min	10-minute blocks since collapsing (144 = gone)

5 Game rules

5.1 Statistics

Stat	RAM	Range	Shown as	Start
Mood (flowers)	\$8E	0–33	×3 (0–99)	random 0–15 each code
IQ	\$8F	0–33	×3	
Stamina (sneaker)	\$90	0–33	×3	
Friendship (people)	\$91	0–33	×3	
Money	\$92	0–99	×\$100	rnd + 15 (≈ \$2400)
Weight	\$8C	8–99	kg	8
Age	\$8D	1–	"YR" (days)	1
Tummy / drinks	\$93 / \$94	0–5	—	0
Sickness	\$95	0–10	—	0
Dirt	\$96	0–	—	0

5.2 Periodic rules

Every 10 minutes of life (not in modes 8, A, B: asleep, collapsed, farewell) [code](#) :

```

if hour ≥ 20 and location ≠ home: go home
care_clock += 1
if care_clock % 4 = 3: # every 30 min
    drinks -= 1; stamina -= 1; care_clock += 1
    if care_clock & 4 = 0: # every 60 min
        if tummy = 0: stamina -= 1; weight -= 1; sickness += 1
        else:
            tummy -= 1; dirt += 1
            if dirt ≥ 3: money -= 2; sickness += 1
if care_clock = $60: room messy (life_bits b3)
if sickness ≥ 5 and ++ill_clock = 6: # every hour while ill
    ill_clock = 0; sickness += 1
    stamina, IQ, mood -= 1; money -= 2; weight -= 1
if sickness ≥ 10 or money = 0: collapse (mode A)

```

Every 24 hours: `age_days++`, stage recomputed, mood, IQ, stamina and friendship -1. If `age_days ≥ lifespan` the farewell starts (mode B), otherwise the birthday scene (mode 9) if the pair is at home awake or asleep.

Attention (!?) is recomputed on every return to sleep (`attention $8E26`). It is shown if tummy = 0, drinks = 0, dirt ≥ 2, sickness ≥ 5, money < 10, the night flag is set, the pair is just waking up (mode `$88`) or the room is messy `code`.

5.3 Care actions

Food (mode 1)

The item code is `base + 16·choice` with base `$10` at home or `$70` in town, plus `$30` for teenagers and adults, and choice ∈ {0,1,2} shown in turn. The middle choice (`$20 $50 $80 $B0`) is a drink: drinks+1, stamina+1, always accepted. `$50` (older, at home) also gives IQ+1, mood-1 and money-1 `code` (a coffee-like drink `inferred`). The other codes are meals: tummy+1, stamina+1, some also weight+1, refused when tummy ≥ 5. The pair refuses to eat at the running track.

Outings (mode 7)

Twelve activity codes, with effects applied by `outing_effect $B67D code`. Names are inferred from the pictures and effects:

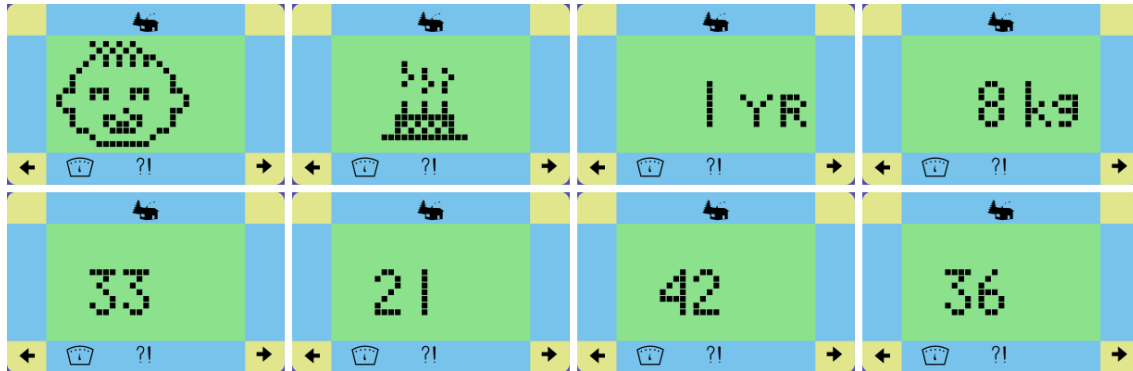
Code	Effect	Name	Code	Effect	Name
\$10	social+1, money-1	TV <code>obs</code>	\$70	stamina+1, tummy-1	sport
\$20	mood+1, money-1	music <code>obs</code>	\$80	stamina+1, drinks-1	sport
\$30	IQ+1, social-1	reading <code>obs</code>	\$90	stamina+1, tummy-1	sport
\$40	IQ+1, money+1	work	\$A0	money+3, stamina-1	hard work
\$50	social+1, money+1	work	\$B0	money-4, mood+2, social+2	party
\$60	mood+1, money+1	work	\$C0	money-2, IQ+2	lessons

Some sub-states include a small challenge checked against data RAM `$10A0`. A wrong answer costs money-1 and shows the refusal animation `code`.

Other actions

Action	Condition	Effect
Clean (5)	dirt ≥ 2, or room messy	dirt = 0 / messy cleared; otherwise refusal <code>code</code>
Doctor (4)	sickness ≥ 5	sickness = 0; stamina, IQ, money, weight -1; if sickness ≥ 6 also mood and money -1 <code>code</code>

Light (2)	night flag (22:00–07:00)	asleep (mode 8), weight+1; by day: refusal <code>code</code>
Games (3)	—	two mini-games (shell game, reels); IQ and stamina up or down with the result <code>observed</code>
Status meter (6)	—	8 pages: stage, age, weight, mood, IQ, stamina, friendship, money <code>code</code>



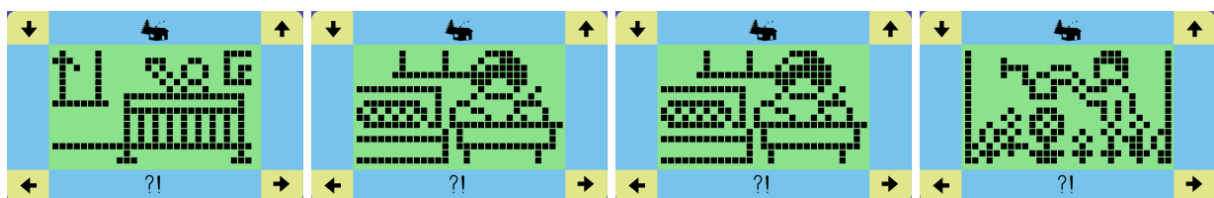
Status meter values of a new pair (stat × 3, money × \$100).

5.4 Day, night and locations

life_bits b6–7 hold the location (0 home, 1 town, 2 running track) and b4–5 the destination while choosing (UP). Babies cannot pick the track. From 18:00 the destination resets to home, and at 20:00 the pair returns home `code`. The night flag is set at 22:00 and cleared at 07:00. Between 23:00 and 07:00 a pair still up falls asleep by itself with weight–1. In mode 8, 07:00–08:00 is the alarm (mode \$88). At 08:00 the pair gets up with weight–1, tummy = drinks = 0 and care_clock = 0.

5.5 Life cycle

Event	Rule
Arrival	clock 12:00, age 1, weight 8, lifespan = 23 + (rnd & 3), stats random <code>code</code>
Stages	first index i with [3, 8, 15, 30][i] ≥ age: baby 1–3, child 4–8, teen 9–15, adult 16+ <code>code</code>
Collapse (A)	sickness ≥ 10 or money = 0. ESC+ENTER within 144 ten-minute blocks (24 h): money = 20, sickness = 0, the four stats –7, hospital (mode 4). After 24 h: mode \$4A, and ESC+ENTER wipes RAM \$85–\$FF (new pair, clock kept) <code>code</code>
Farewell (B)	age ≥ lifespan: portrait, ESC+ENTER → new pair <code>code</code>



Home scenes for the four stages.

5.6 Password, clock stop, test mode

Clock. CLOCK shows the clock (flags_b9 b7). LEFT/RIGHT toggle the sound (flags_b7 b6, bell icon). A long CLOCK sets int_sleep=\$8C (no 2 Hz interrupt): "CLOCK STOP", time and pet frozen. ESC then enters setting (LEFT hour+1, RIGHT minute+1). ENTER restarts at 0 s and opens the password screen `code`.

Password. Two digits stored at \$1090/\$1091. 00 = none, otherwise flags_b9 b3. With a password set, 240 s without a key sets flags_b8 b0, and the next key opens the prompt (mode D). Three wrong entries are counted in link_edges (reused) `code`.

Name and test mode. The intro ends with SIGN NAME (mode \$8C): 10 characters, 0 = space and 1-26 = A-Z, stored at \$1080. ENTER compares them with \$C24F = 13 03 04 00 00 14 05 13 14 00, which is "SCD TEST". A match sets flags_b8 b1 (test mode). On the home screen ENTER then adds a day, ESC adds 10 minutes and a long ESC cycles the stage `code` `observed`.



6 Link protocol

Physical layer

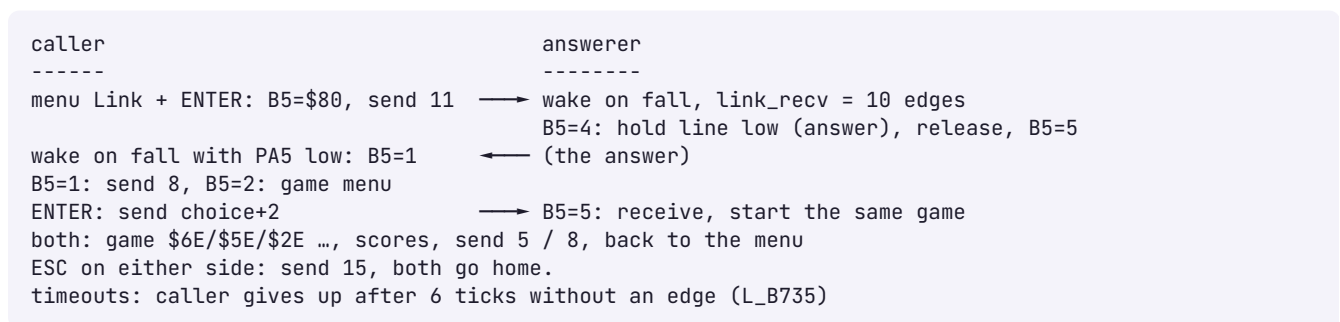
One wire on PA5, idle high through pull-ups. A unit transmits by making PA5 an output (PA_DIR=\$1F) and writing levels, and returns to input (\$3F) to receive. A falling edge wakes a sleeping unit through the key interrupt `code`.

Messages: pulse counts

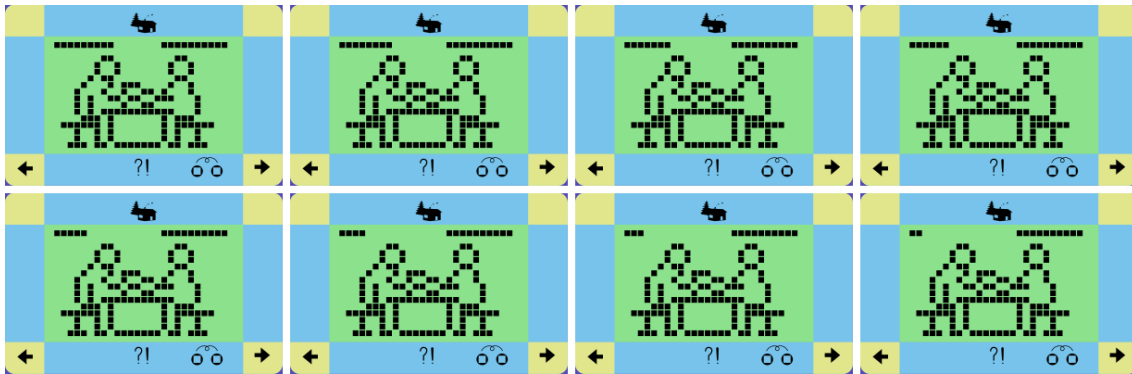
link_send \$BE0A drives the line low, waits ~183 ms, then writes tmp_count levels toggling every ~1.26 ms (88 × 8 cycles), holds ~7.6 ms and releases. link_rcv \$BE41 waits for the line to leave its current level, then counts edges until it has been quiet for 10 × ~0.34 ms. The message is the edge count. Because delay_183ms overwrites tmp, which is the level variable of the send loop, the first write repeats the initial low and the last write is low `code` `observed`.

Sent (tmp_count)	Edges seen	Meaning
11	10 (+ initial fall used as wake-up)	"hello" from the caller (\$B7A5); accepted only if exactly 10 (\$8ED7)
held low ~2.8 s	1 fall	answer: "I'm here" (state 4, \$BA91)
8	—	caller ready / game over, back to the menu
choice + 2	2 / 3 / 4	game chosen: tug-of-war, rock-paper-scissors, arm wrestling
5	—	round finished (IQ, stamina, friendship +1)
15	—	cancel (ESC)

Session state machine (link_state, \$B5)



The latched-edge observation. The answerer's quiet detection (≈ 3.4 ms) ends before the caller releases the line (≈ 7.6 ms hold), so the answer pulse starts while the line is already low. The caller never sees a new falling edge on its pin. With the emulator's edge-triggered key interrupt the handshake fails in **every** case (27/27 timing combinations). It succeeds in every case if a falling edge of the wire is **latched** and delivered once the key interrupt is enabled again, which is common MCU behaviour. The real toys evidently connect, so we believe the SPLB20 latches port-change events. The GBA link bridge implements that model (§7.5) observed inferred.



Tug-of-war on the caller, recorded from two emulated units connected by the simulated wire.

7 Game Boy Advance port

7.1 Overview

The port runs the original ROM on a C re-implementation of the SPLB20 and adds a GBA front end. There is no high-level emulation of game logic in normal play, so behaviour is the toy's own.

Part	File	Runs
SPLB20 core	<code>gba/core/splb20.c</code>	ARM, IWRAM
Emulation driver: pacing, buttons, sound, link	<code>gba/elfin/source/emu.c</code>	ARM, IWRAM, 2048 Hz timer IRQ
LCD renderer	<code>gba/elfin/source/lcd.c</code>	ARM, IWRAM, main loop
Link bridge	<code>gba/core/elfin_link.c</code>	from the IRQ
RTC catch-up	<code>gba/core/elfin_catchup.c</code>	at start-up
RTC driver (S-3511A)	<code>gba/common/rtc.c</code>	—
Front end: menu, saving	<code>gba/elfin/source/main.c</code> , <code>save.c</code>	Thumb, ROM
Assets (generated)	<code>gba/elfin/source/assets.c</code>	ROM data

Memory: ROM image 416 KB; IWRAM 26.1 KB of 32 KB (core 21.4 KB of code, state and tables 4.7 KB, 4 KB kept for the stacks); EWRAM 84 KB (game ROM copy 32 KB, renderer tables, save buffer).

7.2 Core port and verification

`splb20.c` is a line-by-line port of `cores/SPLB20.py`, including its quirks: the 0→1 rule of the stop bits, flag computation order in ADC/SBC, and key interrupts only on falling input changes. Cycle counts are kept in 24.8 fixed point so the fractional sub-clock divider ($560000/32768 = 17 + 23/256$) is exact. Verification:

Test	What it checks	Result
------	----------------	--------

<code>verify_core.py</code>	C and Python cores in lock-step: registers after every instruction, all RAM every 5000, random buttons	18 M instructions (3×6 M) identical, no illegal opcodes
<code>test_sleep_skip.py</code>	sleep fast path vs stepping	identical over 536 s
<code>test_run_equiv.py</code>	<code>splb20_run</code> (all fast paths) vs <code>splb20_step</code> , whole-state compare	identical over 1800 s with random buttons
<code>test_catchup.py</code>	RTC catch-up vs full emulation	clock, life timer, age, lifespan and mode equal for 6, 24, 30, 48 and 100 h
<code>test_link.py</code>	call → answer → game over a simulated wire	27/27 with the bridge (slices 100/273/546 cycles, $\pm 3\%$ skew)

7.3 Performance

A naive interpreter needed about 230 GBA cycles per emulated instruction, and ran the busy intro at 45% speed. Four techniques bring the busiest scenes to full speed, all exact (verified by `test_run_equiv.py`):

- **Sleep skipping.** While the oscillator is stopped, jump straight to the step on which the next timer event is due.
- **Batched timers.** Apply elapsed cycles to the timers once per run of instructions that ends at the next due event. Writes to `$7A/$7C` flush the batch first. No event can be pending inside a batch, so the NMI still lands on the same instruction.
- **Idle-loop skipping.** The ROM is scanned at start-up for the delay loop `85 7F CA D0 FB` and the wait loop `A5 aa C5 bb F0 FA` (aa, bb in RAM). Whole iterations up to the next timer event are skipped and their final register state is set directly.
- **Placement.** ARM code in IWRAM, the game ROM copied to EWRAM, `execute()` inlined into the run loop.

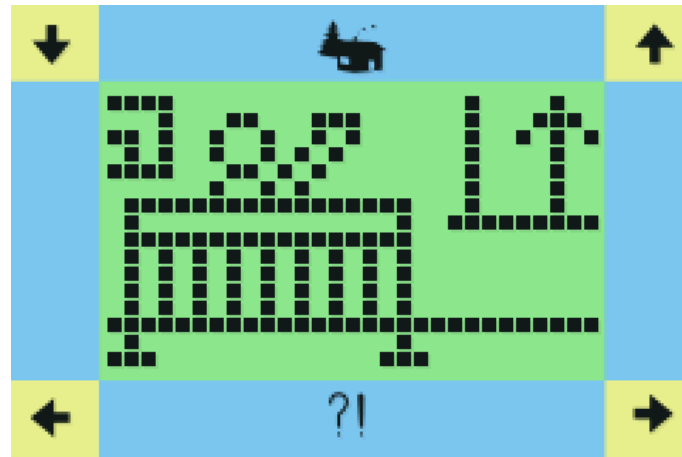
Scene	Emulated instr/s	Speed
Normal life (home)	12–23 k	100.0%
Opening story, busiest part	131 k	96.6%
Opening story, average	110 k	100%

Pacing uses real time. Timer 3 runs freely at 262 144 Hz, and each 2048 Hz interrupt runs the cycles owed for the time that has actually passed ($560000/262144 = 4375/2048$ cycles per tick, accumulated exactly), capped at 1/8 s. A late or merged interrupt is therefore made up rather than lost.

7.4 Display, input, sound

`make_assets.py` renders the SVG panel at 6/11 px per SVG unit, so the 11-unit dot pitch becomes 6 px, with the dots pixel-aligned as 5×5 squares. Every screen pixel touched by a segment or its drop shadow gets a list of contributors (segment, alpha, shadow flag): 19 325 pixels, 28 626 contributions. Segments switch instantly by default. The optional *LCD fade* (START menu) instead moves each segment's brightness 40% towards its target every frame (BrickEmuPy's motion blur of 0.6). It is cosmetic: the ROM never toggles a segment faster than about 20 Hz (51 ms in the opening story, ~1 s in normal play, measured), so it uses no flicker grey levels. Only the pixels of segments whose 16-level brightness changed are recomposed in mode 3. Buttons: A=ENTER, B=ESC, SELECT/L/R=CLOCK, D-pad. UP and DOWN close the same contact pairs as on the toy. The buzzer is mapped to DMG square channel 1. The toy plays `clock / period` Hz with `period = clock_div × (TC_PRESET + 1)`, and the channel plays `131072 / (2048 - rate)` Hz, so the port uses `rate = 2048 - round(131072 · period / clock)`. An earlier build truncated twice and made every note sharp by a different amount (up to +27 cents), which sounded out of tune.

The tunes recorded in the opening story, menus and actions use eight notes `observed` , all periods that are multiples of 32 cycles: $875 \text{ Hz} \times 20/n$ for $n = 20, 16, 13, 12, 11, 10, 9, 8$ (875, 1094, 1346, 1458, 1591, 1750, 1944, 2188 Hz). These are whole-number ratios, not equal temperament, so next to a piano some notes sit up to 36 cents off (E6 +36, B6 -28). That is how the toy itself sounds. On the GBA every one of these notes is within 11 cents of the toy's pitch, the channel's step size at these frequencies. Absolute pitch also depends on the 560 kHz clock estimate `inferred` .



The port in mGBA.

7.5 Link bridge

The GBA link port is used in general-purpose mode (`RCNT = $8000 | dir<<4 | data`). The toy's wire is mapped to `SD`, which the GBA link cable connects to every console (SO/SI are daisy-chained). `SC` can be selected instead. The pin is only ever driven low or released, so two consoles never fight. Once per 2048 Hz slice `elfin_link_before()` feeds the wire to PA5 and `elfin_link_after()` drives the wire when PA5 is output-low. A falling wire edge is latched and delivered with `sp1b20_key_irq()` when the key interrupt is enabled (\$6). A wire never seen high, or low for more than 10 s, is ignored. That protects against missing pull-ups and unconnected cables.

7.6 RTC catch-up

At start-up the port reads the cartridge RTC (Seiko S-3511A protocol on cartridge GPIO `$80000C4-C8`; the ROM contains the `SIIRTC_V` signature that emulators and some loaders look for). It then lets the pet live through the time it was off `code` :

- In the steady modes (home awake, 0; collapsed, A) with the CPU asleep at `$8E83`, a tick that only increments the two half-second counters is applied directly in C.
- Other minute boundaries run `tick_2hz` as a subroutine with a sentinel return address. A `txs` means it escaped into the main program.
- 10-minute and hour boundaries let the game handle the tick completely: the 2 Hz timer fires and the game runs until it is asleep again. That is where bedtime, waking and the other main-program transitions happen.
- Any other mode (animations, birthday, farewell) is emulated normally, with sleep skipping, until steady again.

Validation against full emulation is in §7.2. Cost: about 15–100 k instructions per hour away (100 h $\approx$ 1.5 M), and 13 h 49 min took about 2 s on the GBA. After catching up, the pet clock can be set from the RTC. The player can also cap the time away at one day or pause the pet while off.

7.7 Saving

Battery SRAM (tagged `SRAM_V113`) holds two 8 KB slots, written alternately. Each slot has a header (magic `ELF1`, version, state size, sequence number, RTC seconds since 2000, settings, FNV-1a checksum) and the raw `sp1b20_t`. The header's first word is written last, so a torn write leaves the other slot as the newest valid one. The machine state is copied with interrupts masked and written outside the critical section. Saves happen 3 s after the last key, every minute, and on opening the menu.

7.8 Build and test tooling

`gba/tools/gba.mk` builds with `devkitARM` (`DEVKITARM` set: `gba.specs`, `gbafix`) or with any `arm-none-eabi-gcc` + `newlib`, using the repository's `crt0.s`, `gba.ld` and `gbafix.py`. `libtonc` is vendored and always built from source. Host tools: `mgba_run` (headless `libmgba` runner with a fake RTC, key scripts, screenshots and memory dumps), the verification scripts of §7.2, the workbench, the link simulator, the analysis disassembler and the figure generator used for this document and the manual.

8 Provenance: who made it

The toy has no known packaging, manual or markings that name its maker. The ROM contains two pieces of evidence: a credit left by the developer, and a test program supplied by the chip maker.

8.1 The hidden credit

The 58 bytes at `$FF27-$FF60`, just before the zero padding that runs up to the vector table, are printable text with `$20` added to every character. That offset keeps the text out of a plain `strings` dump. Subtracting `$20` gives `code`:

Sparkle Crystal Development Co.,Ltd. WangLaXian 1996.11.16

stored bytes, and each byte minus `$20`:

\$FF27	73	90	81	92	8B	8C	85	40	63	92	99	93	94	81	8C	40	64	85	96	85
	S	p	a	r	k	l	e		C	r	y	s	t	a	l		D	e	v	e
\$FF3B	8C	8F	90	8D	85	8E	94	40	63	8F	4E	4C	6C	94	84	4E	40	77	81	8E
	l	o	p	m	e	n	t		C	o	.	,	L	t	d	.		W	a	n
\$FF4F	87	6C	81	78	89	81	8E	40	51	59	59	56	4E	51	51	4E	51	56		
	g	L	a	X	i	a	n		1	9	9	6	.	1	1	.	1	6		

Stored bytes (top) and the same bytes minus `$20` (bottom).

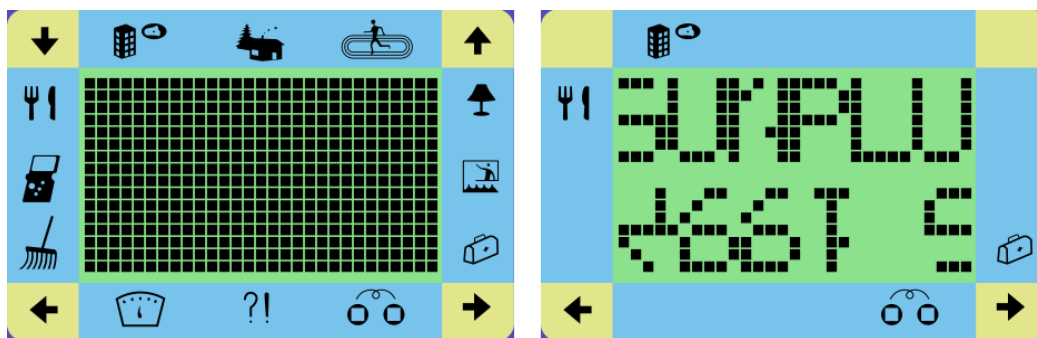
- **Nothing reads it.** The only occurrences of the address `$FF27` in the ROM are three byte pairs inside graphics data (`$D551`, `$EE04`, `$EE40`). No instruction points at it, so the toy never displays it `code`.
- **SCD.** The secret name that enables test mode (`$5.6`), `SCD TEST`, matches the company's initials `inferred`.
- **WangLaXian** reads as a Chinese personal name in pinyin (Wang La-xian), presumably the programmer `inferred`.
- **1996.11.16** is probably the completion or build date. The original Tamagotchi went on sale in Japan on 23 November 1996 `inferred`.

8.2 Sunplus factory test program

The first kilobyte of the ROM, `$8000-$83D4` (followed by `$FF` fill and a 2 KB zero block up to `$8BFF`), is identical byte for byte in all five SPLB20 ROMs in BrickEmuPy's collection (Elfin Twins, Apollo 2 in 1, Apollo Prince & Princess, Big Cat 9 in 1, Gyaoppi 9 in 1). So it belongs to the chip, not to the game. The game never jumps into it. It has its own vectors below the normal ones: test NMI `$FFF2` → `$83BE`, test RESET `$FFF4` → `$8000`, test IRQ `$FFF6` → `$83CE`. This suggests that a test pin selects an alternate vector set **inferred**. After reset it makes port A an input, reads a code applied by the tester, and runs one of seven tests. Each was checked by starting the program at `$8000` on the emulated core with the code applied **observed**:

Code	Routine	Test	Emulated result
<code>\$F0</code>	<code>ft_standby</code>	interrupts off, stop the oscillator (standby current)	<code>SYS_CTRL=\$83</code> , CPU halted
<code>\$FA</code>	<code>ft_lcd_all</code>	all 512 LCD bits on, delay, then 32 kHz LCD-only mode	LCD RAM all <code>\$FF</code> , <code>SYS_CTRL=\$34</code>
<code>\$F5</code>	<code>ft_rom_dump</code>	port A as outputs; stream ROM <code>\$8000-\$FFFF</code> to it forever	port A = <code>78 A2 FF 9A A9...</code>
<code>\$0A</code>	<code>ft_checksum</code>	16-bit ROM sum as nibbles <code>\$1x \$2x \$3x \$4x</code> ; checkerboard test of LCD + CPU RAM (<code>\$F5</code> pass / <code>\$05</code> fail) and data RAM <code>\$1000-\$17FF</code> (<code>\$F8</code> / <code>\$08</code>)	<code>11 21 33 41 F5 F8</code> for sum <code>\$F311</code>
<code>\$0F</code>	<code>ft_ram_echo</code>	the same RAM patterns, every byte read back sent to port A	<code>AA 55 AA 55...</code>
<code>\$AA</code>	<code>ft_lcd_buzzer</code>	64-byte test picture into LCD RAM, buzzer pitch sweep	see figure
<code>\$AF</code>	<code>ft_misc</code>	port/LCD configuration pulses, three buzzer tones, one of each instruction form, then wait for an interrupt	<code>07 00 09</code> (no interrupt, see <code>\$9</code>)

The checksum output has a bug. The fourth nibble should be the high nibble of the sum's high byte (`$4F`), but the code repeats the high nibble of the low byte (`$41`). A tester that compares against a precomputed signature would not notice.



Tests `$FA` and `$AA` on the Elfin Twins panel. The test picture was drawn for another segment layout, but the top row still reads SUNPLU...

8.3 Other toys, and what is not known

Every ROM in BrickEmuPy's `assets/` collection was searched for the credit text under every possible byte offset. Only Elfin Twins has one. The only game data it shares with another dumped toy is its **font**: a pointer table at `$C50B` (28 entries: A-Z, blank, full stop; letter code $n \rightarrow$ entry $n-1$) and 7-byte glyphs at `$C543` (width 5, height 7, 35 pixel bits with the rows stored bottom-up). *Apollo Prince & Princess* stores its font in the same format at `$FF0B`. 16 of its 26 letters are identical to Elfin's and 10 are drawn differently. The identical ones are the letters with only one natural 5×7 shape. That points to shared tools or conventions, not proof of a shared developer **inferred**.



The two fonts, decoded from the ROMs. Letters drawn differently are in orange.

Web searches (English and Chinese) found no trace of Sparkle Crystal Development Co., Ltd., of WangLaXian, or of other products. Leads for further research: the packaging of a boxed GM-021 (distributor name), company registries (Hong Kong, Taiwan), and collectors of 1997 virtual pets.

9 Open questions

- **Latched key edges.** The link model of §6 should be confirmed on real hardware, for example by linking a real toy to the GBA port, or by logging PA5 of two real toys.
- **SD pull-up on the GBA link cable in GPIO mode.** `elfin-linktest.gba` reports whether a released SD idles high. If not, the port can use SC.
- **Names of some items.** Food items and outings are identified by effect. Their names are inferred from the pictures.
- **Mini-game scoring.** The exact rules of the two games in mode 3 and of the outing challenges were only observed, not fully traced.
- **Some flag bits** in `flags_b7/b8/b9` and `sub_sel` are only partly understood. See the annotated disassembly.
- **Who Sparkle Crystal Development was** (§8.3), and how the SPLB20 enters its test mode (a test pin is assumed; the test-mode interrupt vectors are not emulated, so the interrupt part of test `$AF` was not checked).
- **Clock accuracy.** BrickEmuPy's 560 kHz is an estimate for the RC oscillator. The 2 Hz tick is derived from the 32 kHz crystal, so game time is exact either way.

A Appendix: routine index

Addr	Name	What it does
\$8000	<code>ft_reset</code>	FACTORY TEST (Sunplus): entry from test vector \$FFF4. Port A as inputs, then read the test code the tester applies: \$F0 standby \$FA LCD all on \$F5 ROM dump \$AF misc/instr/IRQ \$0A checksum + RAM test \$0F RAM pattern echo \$AA LCD pattern + buzzer Anything else: clear the watchdog and read again.
\$804B	<code>ft_standby</code>	Test \$F0: interrupts off, stop the oscillator (standby current)
\$8063	<code>ft_lcd_all</code>	Test \$FA: every LCD segment on, wait, then 32 kHz / LCD-only mode
\$8094	<code>ft_rom_dump</code>	Test \$F5: port A as outputs, stream ROM \$8000-\$FFFF to it forever

Addr	Name	What it does
\$80BB	ft_misc	Test \$AF: port/LCD config pulses, three buzzer tones, execute one of each instruction form, then wait for an interrupt (result on port A: \$09 = none, or the pending flags << 4 \$09 from ft_nmi)
\$8171	ft_checksum	Test \$0A: 16-bit sum of all ROM bytes, sent as four nibbles \$1x \$2x \$3x \$4x; then checkerboard \$AA/\$55 over LCD RAM + CPU RAM (port A \$F5 = pass, \$05 = fail) and over data RAM \$1000-\$17FF (\$F8 = pass, \$08 = fail). Emulated: 11 21 33 41 F5 F8 for sum \$F311 (the 4th nibble repeats the low byte's high nibble instead of the high byte's - a bug in Sunplus's code).
\$82A8	ft_ram_echo	Test \$0F: same RAM patterns, every byte read back is output on port A
\$8344	ft_lcd_buzzer	Test \$AA: copy the 64-byte pattern ft_lcd_pattern to LCD RAM, sweep the buzzer
\$83BE	ft_nmi	Test-mode NMI (vector \$FFF2): pending flags << 4 \$09 to port A, mark done
\$83CE	ft_irq	Test-mode IRQ (vector \$FFF6): return
\$83CF	ft_delay	~0.5 ms delay loop
\$8C00	irq	IRQ vector (unused: rti)
\$8C01	nmi	NMI handler (interrupts while the CPU is running)
\$8C6D	tick_2hz	2 Hz tick: clock, life timer, 10-minute and daily events
\$8DE7	night_flag	Set/clear the night flag (22:00 / 07:00)
\$8E09	growth_stage	Stage from age: <=3 baby, <=8 child, <=15 teen, else adult
\$8E26	attention	Decide whether the ?! icon is shown
\$8E74	sleep_main	Common exit: update ?!, wait for the melody, then sleep
\$8E7A	sleep	Stop the oscillator until the next interrupt (32 kHz keeps running)
\$8E83	snd_play	Start melody X (also the instruction right after the STOP)
\$8EB2	wake	Wake-up handler: link line, keys, tick, animation
\$8FB8	key_decode	Debounce and decode the five contacts into a key code
\$9061	gfx_draw	Draw a bitmap from ROM (\$A2) at (\$A0,\$A1) into LCD RAM
\$9338	rnd_next	Advance the random number generator
\$93AD	key_dispatch	Per-key handler: jump table T_959D by mode
\$9520	tick_dispatch	Per-tick handler: jump table T_957F by mode
\$95BB	password_tick	Mode D (password) tick
\$9631	password_key	Mode D keys
\$970C	reset	Reset / wake-up vector. After STOP the chip restarts here; pending interrupt flags decide between cold start and wake-up.
\$9827	new_pair	Cold start: clock 12:00, lifespan, stats, intro
\$98E8	home_tick	Mode 0 (home) tick: alerts, idle animations, bedtime
\$9AF6	home_key	Mode 0 (home) keys: menu, destination, test mode
\$9CCA	test_add_10min	TEST MODE: ESC adds 10 minutes
\$9CF1	test_stage	TEST MODE: long ESC cycles the growth stage
\$9D51	cursor_off	Hide the menu cursor icon
\$9D83	cursor_on	Show the menu cursor icon
\$9DBA	attention_on	Light ?! (with beep)
\$9DC8	attention_off	Clear ?!
\$9E46	drinks_dec	drinks - 1
\$9E4D	tummy_dec	tummy - 1
\$9E54	stamina_dec	stamina - 1
\$9E5B	iq_dec	IQ - 1
\$9E62	mood_dec	mood - 1
\$9E69	social_dec	friendship - 1
\$9E70	money_dec	money - \$100; at \$0 the pair collapses

Addr	Name	What it does
\$9E7E	weight_dec	weight - 1 kg (not below 8)
\$9E87	drinks_inc	drinks + 1 (max 5)
\$9E90	tummy_inc	tummy + 1 (max 5)
\$9E99	stamina_inc	stamina + 1 (max 33)
\$9EA2	iq_inc	IQ + 1 (max 33)
\$9EAB	mood_inc	mood + 1 (max 33)
\$9EB4	social_inc	friendship + 1 (max 33)
\$9EBD	money_inc	money + \$100 (max \$9900)
\$9EC6	weight_inc	weight + 1 kg (max 99)
\$A007	food_tick	Mode 1 (food) tick
\$A07C	food_key	Mode 1 (food) keys
\$A22B	food_effect	Apply the effect of the served food
\$A287	lamp_tick	Mode 2 (light) tick
\$A386	game_tick	Mode 3 (games) tick
\$A40A	game_key	Mode 3 (games) keys
\$AA8E	doctor_tick	Mode 4 (doctor) tick
\$AAED	clean_tick	Mode 5 (clean) tick
\$AB3A	meter_tick	Mode 6 (status meter) tick
\$AB84	meter_key	Mode 6 keys
\$AFAD	outing_tick	Mode 7 (outings) tick
\$B0B9	outing_key	Mode 7 keys
\$B67D	outing_effect	Apply the effect of outing code 1-12
\$B76C	link_tick	Mode E (link) tick: session state machine
\$B7ED	link_key	Mode E keys
\$BE0A	link_send	Send tmp_count level changes on PA5 (link cable)
\$BE41	link_recv	Count PA5 edges until the line is quiet
\$BE8F	delay_8ms	6 x delay_1ms
\$BEA1	delay_1ms	~1.26 ms delay (88 x 8 cycles)
\$BEAD	delay_short	~0.3 ms delay (20 x 8 cycles)
\$BF12	sleep_tick	Mode 8 (asleep) tick: alarm 07:00, up 08:00
\$BFED	birthday_tick	Mode 9 (birthday) tick
\$C015	collapse_tick	Mode A (collapsed) tick: 24 h to rescue
\$C09B	collapse_key	Mode A keys: ESC+ENTER rescue / new pair
\$C103	ending_tick	Mode B (farewell) tick
\$C154	wipe_ram	Clear RAM \$85-\$FF and start a new pair
\$C2CB	intro_tick	Mode C (opening story) tick
\$C479	delay_183ms	delay_n with A=\$32 (clobbers \$AF)
\$C47D	delay_n	delay \$AF x 256 x 8 cycles
\$C61B	wait_tick	Busy-wait until the next 2 Hz tick
\$C626	wait_melody	Busy-wait until the melody has finished
\$C637	lcd_fill	Fill LCD RAM with A
\$C641	anim_start	Start animation / scene \$C4
\$C72A	clock_screen	Show the clock screen

B Appendix: LCD mapping

Dot (x, y) of the 29×16 matrix $\rightarrow$ LCD RAM byte (hex) and bit (subscript).

	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19	20	21	22	23	24	25	26	27	28
0	1F ₁	1F ₂	1F ₃	1F ₄	1F ₅	1F ₆	1F ₇	1E ₀	1E ₁	1E ₂	1E ₃	1E ₄	1E ₅	1E ₆	1E ₇	1D ₀	1D ₁	1D ₂	1D ₃	1D ₄	1D ₅	1D ₆	1D ₇	1C ₀	1C ₁	1C ₂	1C ₃	1C ₄	1C ₅
1	17 ₁	17 ₂	17 ₃	17 ₄	17 ₅	17 ₆	17 ₇	16 ₀	16 ₁	16 ₂	16 ₃	16 ₄	16 ₅	16 ₆	16 ₇	15 ₀	15 ₁	15 ₂	15 ₃	15 ₄	15 ₅	15 ₆	15 ₇	14 ₀	14 ₁	14 ₂	14 ₃	14 ₄	14 ₅
2	0F ₁	0F ₂	0F ₃	0F ₄	0F ₅	0F ₆	0F ₇	0E ₀	0E ₁	0E ₂	0E ₃	0E ₄	0E ₅	0E ₆	0E ₇	0D ₀	0D ₁	0D ₂	0D ₃	0D ₄	0D ₅	0D ₆	0D ₇	0C ₀	0C ₁	0C ₂	0C ₃	0C ₄	0C ₅
3	07 ₁	07 ₂	07 ₃	07 ₄	07 ₅	07 ₆	07 ₇	06 ₀	06 ₁	06 ₂	06 ₃	06 ₄	06 ₅	06 ₆	06 ₇	05 ₀	05 ₁	05 ₂	05 ₃	05 ₄	05 ₅	05 ₆	05 ₇	04 ₀	04 ₁	04 ₂	04 ₃	04 ₄	04 ₅
4	27 ₁	27 ₂	27 ₃	27 ₄	27 ₅	27 ₆	27 ₇	26 ₀	26 ₁	26 ₂	26 ₃	26 ₄	26 ₅	26 ₆	26 ₇	25 ₀	25 ₁	25 ₂	25 ₃	25 ₄	25 ₅	25 ₆	25 ₇	24 ₀	24 ₁	24 ₂	24 ₃	24 ₄	24 ₅
5	2F ₁	2F ₂	2F ₃	2F ₄	2F ₅	2F ₆	2F ₇	2E ₀	2E ₁	2E ₂	2E ₃	2E ₄	2E ₅	2E ₆	2E ₇	2D ₀	2D ₁	2D ₂	2D ₃	2D ₄	2D ₅	2D ₆	2D ₇	2C ₀	2C ₁	2C ₂	2C ₃	2C ₄	2C ₅
6	37 ₁	37 ₂	37 ₃	37 ₄	37 ₅	37 ₆	37 ₇	36 ₀	36 ₁	36 ₂	36 ₃	36 ₄	36 ₅	36 ₆	36 ₇	35 ₀	35 ₁	35 ₂	35 ₃	35 ₄	35 ₅	35 ₆	35 ₇	34 ₀	34 ₁	34 ₂	34 ₃	34 ₄	34 ₅
7	3F ₁	3F ₂	3F ₃	3F ₄	3F ₅	3F ₆	3F ₇	3E ₀	3E ₁	3E ₂	3E ₃	3E ₄	3E ₅	3E ₆	3E ₇	3D ₀	3D ₁	3D ₂	3D ₃	3D ₄	3D ₅	3D ₆	3D ₇	3C ₀	3C ₁	3C ₂	3C ₃	3C ₄	3C ₅
8	38 ₃	38 ₂	38 ₁	38 ₀	39 ₇	39 ₆	39 ₅	39 ₄	39 ₃	39 ₂	39 ₁	39 ₀	3A ₇	3A ₆	3A ₅	3A ₄	3A ₃	3A ₂	3A ₁	3A ₀	3B ₇	3B ₆	3B ₅	3B ₄	3B ₃	3B ₂	3B ₁	3B ₀	3C ₇
9	30 ₃	30 ₂	30 ₁	30 ₀	31 ₇	31 ₆	31 ₅	31 ₄	31 ₃	31 ₂	31 ₁	31 ₀	32 ₇	32 ₆	32 ₅	32 ₄	32 ₃	32 ₂	32 ₁	32 ₀	33 ₇	33 ₆	33 ₅	33 ₄	33 ₃	33 ₂	33 ₁	33 ₀	34 ₇
10	28 ₃	28 ₂	28 ₁	28 ₀	29 ₇	29 ₆	29 ₅	29 ₄	29 ₃	29 ₂	29 ₁	29 ₀	2A ₇	2A ₆	2A ₅	2A ₄	2A ₃	2A ₂	2A ₁	2A ₀	2B ₇	2B ₆	2B ₅	2B ₄	2B ₃	2B ₂	2B ₁	2B ₀	2C ₇
11	20 ₃	20 ₂	20 ₁	20 ₀	21 ₇	21 ₆	21 ₅	21 ₄	21 ₃	21 ₂	21 ₁	21 ₀	22 ₇	22 ₆	22 ₅	22 ₄	22 ₃	22 ₂	22 ₁	22 ₀	23 ₇	23 ₆	23 ₅	23 ₄	23 ₃	23 ₂	23 ₁	23 ₀	24 ₇
12	00 ₃	00 ₂	00 ₁	00 ₀	01 ₇	01 ₆	01 ₅	01 ₄	01 ₃	01 ₂	01 ₁	01 ₀	02 ₇	02 ₆	02 ₅	02 ₄	02 ₃	02 ₂	02 ₁	02 ₀	03 ₇	03 ₆	03 ₅	03 ₄	03 ₃	03 ₂	03 ₁	03 ₀	04 ₇
13	08 ₃	08 ₂	08 ₁	08 ₀	09 ₇	09 ₆	09 ₅	09 ₄	09 ₃	09 ₂	09 ₁	09 ₀	0A ₇	0A ₆	0A ₅	0A ₄	0A ₃	0A ₂	0A ₁	0A ₀	0B ₇	0B ₆	0B ₅	0B ₄	0B ₃	0B ₂	0B ₁	0B ₀	0C ₇
14	10 ₃	10 ₂	10 ₁	10 ₀	11 ₇	11 ₆	11 ₅	11 ₄	11 ₃	11 ₂	11 ₁	11 ₀	12 ₇	12 ₆	12 ₅	12 ₄	12 ₃	12 ₂	12 ₁	12 ₀	13 ₇	13 ₆	13 ₅	13 ₄	13 ₃	13 ₂	13 ₁	13 ₀	14 ₇
15	18 ₃	18 ₂	18 ₁	18 ₀	19 ₇	19 ₆	19 ₅	19 ₄	19 ₃	19 ₂	19 ₁	19 ₀	1A ₇	1A ₆	1A ₅	1A ₄	1A ₃	1A ₂	1A ₁	1A ₀	1B ₇	1B ₆	1B ₅	1B ₄	1B ₃	1B ₂	1B ₁	1B ₀	1C ₇

C Appendix: instruction set

The 70 opcodes implemented by the SPLB20 core, with cycle counts from `cores/SPLB20.py`.

Op	Mn	Mode	Cycles	Op	Mn	Mode	Cycles
\$00	brk	impl	7	\$86	stx	zp	3
\$05	ora	zp	3	\$8A	txa	impl	2
\$08	php	impl	3	\$8E	stx	abs	4
\$09	ora	#imm	2	\$90	bcc	rel	2 / 3 (+1 page)
\$10	bpl	rel	2 / 3 (+1 page)	\$95	sta	zp,X	4
\$18	clc	impl	2	\$9A	txs	impl	2
\$20	jsr	abs	6	\$A1	lda	(zp,X)	6
\$24	bit	zp	3	\$A2	ldx	#imm	2
\$25	and	zp	3	\$A5	lda	zp	3
\$26	rol	zp	5	\$A6	ldx	zp	3
\$28	plp	impl	4	\$A9	lda	#imm	2
\$29	and	#imm	2	\$AA	tax	impl	2
\$2A	rol	A	2	\$AD	lda	abs	4
\$2C	bit	abs	4	\$AE	ldx	abs	4
\$30	bmi	rel	2 / 3 (+1 page)	\$B0	bcs	rel	2 / 3 (+1 page)
\$38	sec	impl	2	\$B5	lda	zp,X	4
\$40	rti	impl	6	\$B8	clv	impl	2
\$45	eor	zp	3	\$BA	tsx	impl	2
\$48	pha	impl	3	\$BD	lda	abs,X	?
\$49	eor	#imm	2	\$C5	cmp	zp	3
\$4C	jmp	abs	3	\$C6	dec	zp	5

Op	Mn	Mode	Cycles
\$50	bvc	rel	2 / 3 (+1 page)
\$55	eor	zp,X	4
\$58	cli	impl	2
\$60	rts	impl	6
\$65	adc	zp	3
\$66	ror	zp	5
\$68	pla	impl	4
\$69	adc	#imm	2
\$6A	ror	A	2
\$6C	jmp	(ind)	6
\$70	bvs	rel	2 / 3 (+1 page)
\$78	sei	impl	2
\$81	sta	(zp,X)	6
\$85	sta	zp	3

Op	Mn	Mode	Cycles
\$C9	cmp	#imm	2
\$CA	dex	impl	2
\$D0	bne	rel	2 / 3 (+1 page)
\$D5	cmp	zp,X	4
\$D6	dec	zp,X	6
\$E0	cpx	#imm	2
\$E4	cpx	zp	3
\$E5	sbc	zp	3
\$E6	inc	zp	5
\$E8	inx	impl	2
\$E9	sbc	#imm	2
\$EA	nop	impl	2
\$F0	beq	rel	2 / 3 (+1 page)
\$F8	sed	impl	2